

Bilgisayarla Görü Dersi – Ödev 4: HOG (Histogram of Oriented Gradients)

Öğrenci Adı Soyadı: Fazlı Efe Önder

Öğrenci No: 220212033

Bölüm: Yapay Zekâ Mühendisliği

Üniversite: Ostim Teknik Üniversitesi

Tarih: 05.12.2025

Özet (Abstract)

Bu çalışmada, Histogram of Oriented Gradients (HOG) özellik çıkarım yöntemi kullanılarak üç farklı problem ele alınmıştır: (1) HOG'un temel bileşenlerinin sıfırdan implementasyonu ve görselleştirilmesi, (2) hazır ve özel eğitilmiş HOG + SVM modelleriyle nesne tespiti (insan tespiti ve seçilen özel nesne), (3) HOG özellikleriyle çalışan bir görüntü sınıflandırma sisteminin kurulması ve farklı makine öğrenmesi yöntemlerinin karşılaştırılması.

İlk problemde, gradyan hesaplama, hücre histogramlarının oluşturulması, blok normalizasyonu ve tam HOG descriptor vektörünün elde edilmesi adımları manuel olarak kodlanmış ve çeşitli test görüntüleri üzerinde görselleştirme yapılmıştır. İkinci problemde, OpenCV'nin hazır insan dedektörü ile yaya tespiti gerçekleştirilmiş, daha sonra sliding window yaklaşımı ile seçilen bir nesne kategorisi için özel bir HOG + Linear SVM dedektörü eğitilmiştir. Üçüncü problemde ise, HOG özellikleri kullanılarak Linear SVM, RBF çekirdekli SVM ve k-NN ($k=5$) sınıflandırıcıları eğitilmiş ve doğruluk değerleri karşılaştırılmıştır.

DeneySEL sonuçlar, HOG özelliklerinin özellikle insan tespiti ve ikili sınıflandırma problemlerinde başarılı sonuçlar verdiğini göstermektedir. Ayrıca, eşik değerleri ve NMS parametreleri gibi hiperparametrelerin tespit performansını önemli ölçüde etkilediği gözlemlenmiştir.

1. Giriş

Bu ödevde amaç, bilgisayarla görü alanında yaygın kullanılan bir özellik çıkarım yöntemi olan Histogram of Oriented Gradients (HOG) algoritmasını hem teorik hem de pratik açıdan anlamak ve gerçek dünya problemlerine uygulamaktır. HOG, ilk olarak Dalal ve

Triggs tarafından insan tespiti için önerilmiş ve daha sonra birçok nesne tespiti ve sınıflandırma probleminde standart bir yöntem hâline gelmiştir.

HOG'un temel fikri, bir görüntüdeki yerel gradyan yönelimlerinin histogramlarını kullanarak nesnenin şekil bilgisini temsil etmektir. Kenar ve köşe bölgeleri, nesnenin yapısını taşıyan en önemli alanlar olduğu için, bu bölgelerdeki gradyan büyüklükleri ve yönelimleri güçlü bir ayırıştırıcı özellik kümesi sunar.

Bu rapor kapsamında üç ana problem çözülmüştür:

- **Problem 1:** HOG özellik çıkarımını sıfırdan implement etmek ve görselleştirmek.
- **Problem 2:** HOG tabanlı nesne tespiti (önce hazır insan dedektörü, sonra özel nesne dedektörü).
- **Problem 3:** HOG özellikleriyle çalışan bir sınıflandırma sistemi kurmak ve farklı sınıflandırıcıları karşılaştırmak.

Böylece hem düşük seviyeli görüntü işleme adımları (gradyan, histogram, normalizasyon) hem de üst seviye makine öğrenmesi adımları (SVM, k-NN, model kaydetme, değerlendirme) bir arada deneyimlenmiştir.

2. Metodoloji

2.1. Problem 1 – HOG Özellik Çıkarımı

Problem 1'de HOG algoritmasının bileşenleri hazır kütüphaneler kullanılmadan, adım adım implement edilmiştir. Aşağıdaki fonksiyonlar geliştirilmiştir:

- `compute_gradients(image)`
 - Giriş: Gri tonlamalı görüntü.
 - Çıkış: Gradyan büyüklüğü ve yönü (magnitude, angle).
 - Yöntem: x ve y yönlerinde [-1, 0, 1] filtresi ile konvolüsyon veya `np.gradient` kullanılarak Gx ve Gy hesaplandı. Ardından
 - $|G| = \sqrt{Gx^2 + Gy^2}$
 - $\theta = \arctan2(Gy, Gx)$formülleri ile büyüklük ve açı elde edildi.
- `create_cell_histogram(cell_magnitude, cell_angle, num_bins=9)`
 - Her hücre (örneğin 8×8 piksel) için, 0–180° aralığı 9 eşit bin'e bölündü.
 - Her pikselin gradyan büyüklüğü, açıya karşılık gelen bin'e ağırlık olarak eklendi.

- Sınırdaki kalan açılar için iki komşu bin arasında lineer interpolasyon uygulanarak daha yumuşak bir dağılım sağlandı.
- `normalize_block(block_histogram, method="L2")`
 - 2×2 hücreden oluşan bloklar birleştirilerek blok histogramı oluşturuldu.
 - L2 norm ile normalizasyon uygulandı:

$$h_{norm} = \frac{h}{\sqrt{\|h\|^2 + \epsilon^2}}$$

- Blok normalizasyonu, aydınlatma değişimlerine karşı daha dayanıklı bir özellik vektörü elde etmek için kullanıldı.
- `compute_hog_descriptor(image, cell_size=(8,8), block_size=(2,2), num_bins=9)`
 - Görüntü önce hücrelere (cells) ayrıldı.
 - Her hücre için yönelim histogramları hesaplandı.
 - Bloklar (ör. 2×2 hücre) sliding window ile tüm görüntü üzerinde dolaştırıldı.
 - Her blok için normalize edilmiş histogramlar çıkarıldı ve tüm bloklar birleştirilerek final HOG descriptor vektörü elde edildi.
- `visualize_hog(image, cell_size=(8,8), num_bins=9)`
 - Her hücre için en baskın bin yönünde, histogram değeriyle orantılı uzunlukta çizgiler çizilerek HOG görselleştirmesi oluşturuldu.
 - Orijinal görüntü ile HOG görselleştirmesi yan yana gösterildi.

Bu fonksiyonlar, basit geometrik şekiller (kare, daire, üçgen), kenarları belirgin nesneler ve insan silüetleri gibi en az 5 farklı test görüntüsü üzerinde denenmiş, elde edilen descriptor boyutları ve görselleştirmeler incelenmiştir.

2.2. Problem 2 – Nesne Tespiti

Problem 2, iki alt başlıktan oluşmaktadır: hazır insan dedektörü ile yaya tespiti ve özel bir nesne için sliding window + HOG + SVM tabanlı dedektör.

2.2.1. İnsan (Yaya) Tespiti – OpenCV HOG + SVM

Bu kısımda OpenCV'nin önceden eğitilmiş HOG + SVM insan dedektörü kullanılmıştır. Kod tarafında:

- `cv2.HOGDescriptor()` nesnesi oluşturuldu.
- `hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())` ile hazır insan modeli yüklendi.

- Test görüntüleri:
 - veri/Train/JPEGImages altında insan1.png – insan40.png (insan içeren)
 - Aynı klasörde nesne1.jpg – nesne40.jpg (insan içermeyen)
- Çok ölçekli tespit için hog.detectMultiScale fonksiyonu kullanıldı:

```
rects, weights = hog.detectMultiScale(
    image,
    hitThreshold=hit_threshold,
    winStride=(8, 8),
    padding=(8, 8),
    scale=1.05
)
```

- Her tespit için weights listesinden gelen skor, confidence score olarak kullanıldı.
- Düşük skorlu tespitleri elemek için minimum skor eşiği (min_confidence) uygulandı.
- Çakışan bounding box'ları birleştirmek için Non-Maximum Suppression (NMS) fonksiyonu kullanıldı (IoU eşiği yaklaşık 0.5 alınmıştır).

Sonuçlar, veri/pedestrians_results klasörüne det_*.png/jpg olarak kaydedilmiştir.

2.2.2. Özel Nesne Tespiti – Sliding Window + HOG + Linear SVM

Bu bölümde, seçilen bir nesne kategorisi (örneğin belirli bir nesne/obje sınıfı) için kendi dedektörümüz eğitilmiştir.

- **Veri toplama:**
 - veri/custom/train_pos klasöründe en az 50 pozitif patch (nesne içeren).
 - veri/custom/train_neg klasöründe en az 50 negatif patch (nesne içermeyen).
 - Tüm görüntüler HOG penceresi boyutuna (64×128) yeniden boyutlandırılmıştır.
- **HOG özellik çıkarımı:**
 - OpenCV'nin cv2.HOGDescriptor sınıfı şu parametrelerle kullanılmıştır:
 - winSize = (64, 128)
 - blockSize = (16, 16)
 - blockStride = (8, 8)

- `cellSize = (8, 8)`
 - `nbins = 9`
- **Sınıflandırıcı eğitimi:**
 - Pozitif ve negatif HOG vektörleri birleştirilerek X ve y matrisleri oluşturulmuştur.
 - `sklearn.svm.LinearSVC` ile ikili sınıflandırıcı eğitilmiştir.
 - Eğitim tamamlandıktan sonra model disk'e kaydedilmiştir (`joblib.dump`).
- **Sliding window tespiti:**
 - Test görüntüleri `veri/custom/test_images` klasöründen okunmuştur.
 - Gri seviyeye çevrilen görüntü, önce orijinal çözünürlükte, daha sonra ölçek küçültülerek (örneğin her adımda 1.2 ile bölünerek) çok ölçekli olarak taranmıştır.
 - Her ölçekte, (8,8) adımlarla sliding window ile pencereler gezilmiş, her pencerenin HOG'u çıkarılıp Linear SVM ile skor hesaplanmıştır.
 - SVM skoru belirli bir eşiğin (`DECISION_THRESHOLD`) üzerinde ise pencere pozitif tespit olarak kaydedilmiştir.
 - Tüm pozitif pencerelere NMS uygulanmış, çakışan kutular birleştirilmiş ve son tespitler görüntü üzerine çizilmiştir.

Sonuçlar `veri/custom/results` klasörüne `det_*.png/jpg` olarak kaydedilmiştir.

2.3. Problem 3 – HOG ile Sınıflandırma ve Karşılaştırma

Problem 3'te, HOG özellik vektörleri kullanılarak bir görüntü sınıflandırma sistemi kurulmuş ve farklı sınıflandırıcılar karşılaştırılmıştır. Tüm işlemler `src/classification.py` dosyasında gerçekleştirilmiştir.

- **Veri seti:**
 - Pozitif sınıf: `veri/custom/train_pos`
 - Negatif sınıf: `veri/custom/train_neg`
 - Her bir patch HOG penceresi boyutuna (64×128) getirildikten sonra HOG vektörü çıkarılmıştır.
 - Pozitif ve negatif örnekler birleştirilerek X (özellik matrisi) ve y (etiket vektörü) oluşturulmuştur.
- **Eğitim/Test bölünmesi:**
 - Veri seti, `train_test_split` ile %70 eğitim / %30 test olacak şekilde bölünmüştür.
 - Stratified bölme kullanılarak sınıf dağılımı korunmuştur.
- **Kullanılan sınıflandırıcılar:**

- Linear SVM (LinearSVC)
- RBF çekirdekli SVM (SVC(kernel="rbf"))
- k-En Yakın Komşu (KNeighborsClassifier(n_neighbors=5))
- **Değerlendirme metrikleri:**
 - Accuracy (doğruluk)
 - Confusion matrix
 - Precision, recall ve F1-score içeren classification report
- **Model kaydetme:**
 - Elde edilen modeller arasında en yüksek test doğruluğuna sahip olan seçilmiştir.
 - Bu “en iyi model”, ödev gereği **models/trained_classifier.pkl** dosyasına `joblib.dump` ile kaydedilmiştir.
- **Grafik:**
 - Modellerin doğruluk değerleri bir listeye toplanmış,
 - `matplotlib` ile bar grafiği çizilmiş ve
 - `report/figures/classification_accuracy.png` adıyla kaydedilmiştir.

3. Deneysel Sonuçlar

3.1. Problem 1 Sonuçları

Problem 1 kapsamında:

- Farklı tipte en az 5 test görüntüsü üzerinde HOG hesaplanmıştır.
- Her görüntü için:
 - Orijinal görüntü
 - HOG görselleştirmesi
 - HOG vektörünün boyutu elde edilmiştir.

Örneğin, 64×128 boyutlu bir görüntü için kullanılan parametrelerle yaklaşık 3780 boyutlu bir HOG vektörü elde edilmiştir (bu değer hücre, blok ve bin sayılarına bağlıdır).

Farklı `cell_size` ve `num_bins` değerleri denenmiş, daha küçük hücre boyutlarının daha detaylı fakat daha yüksek boyutlu özellik vektörleri ürettiği, daha az bin kullanılan durumlarda ise yönelim bilgisinin daha kaba ama daha küçük boyutlu bir vektöre dönüştüğü gözlemlenmiştir.

3.2. Problem 2 Sonuçları

3.2.1. İnsan Tespiti (Pedestrian Detection)

Toplamda **40 insan içeren** (insan*.png) ve **40 insan içermeyen** (nesne*.jpg) görüntü üzerinde test yapılmıştır. Örnek konsol çıktısı:

- Toplam görüntü sayısı: 80
- Toplam tespit sayısı: 32
- Görüntü başına ortalama tespit: 0.40

İnsan görüntülerinde çoğu durumda en az bir kişi doğru tespit edilirken, nesne görüntülerinde bazı false positive tespitler gözlemlenmiştir. Özellikle karmaşık arka planlı veya dikey çizgilere sahip sahnelerde, model insan olmayan bölgeleri de insan olarak algılayabilmektedir.

Farklı hit_threshold ve min_confidence değerleri denenerek şu eğilim görülmüştür:

- **Düşük eşik** → daha fazla tespit, ancak false positive sayısı yüksek.
- **Orta eşik** → tespit sayısı azaltırken, false positive'leri de azaltır ve daha dengeli sonuç verir.
- **Yüksek eşik** → false positive'ler belirgin şekilde azalır ancak bazı gerçek insanlar kaçırılmaya (false negative) başlanır.

Bu sonuçlar, raporda insan tespiti örnek görselleri ve false positive/negative örnekleri üzerinden tartışılmıştır.

3.2.2. Özel Nesne Tespiti

Özel nesne tespitinde kullanılan Linear SVM modeli, pozitif ve negatif patch'ler üzerinde eğitildikten sonra 10 farklı test görüntüsü üzerinde denenmiştir. İlk denemelerde decision threshold düşük tutulduğunda:

- Kalabalık sahnelerde (örneğin crop001638.png, crop001639.png, crop001641.png) çok sayıda (yüzlerce) tespit oluştuğu,
- Bu tespitlerin bir kısmının gerçek nesne, çoğunun ise false positive olduğu

gözlemlenmiştir.

Daha sonra:

- Decision threshold değeri yükseltilerek düşük skorlu pencereler elenmiş,

- NMS IoU eşiği ayarlanarak üst üste binen kutular azaltılmıştır.

Böylece, daha az ama daha güvenilir tespitler elde edilmiştir. Yine de, veri setinin küçüklüğü, pozitif/negatif örneklerin çeşitliliği ve seçilen nesnenin zorluğu gibi faktörler nedeniyle, bazı sahnelerde hâlâ false positive ve false negative'ler mevcuttur.

3.3. Problem 3 Sonuçları

Problem 3'te aynı HOG özellikleri kullanılarak üç farklı sınıflandırıcı eğitilmiştir. Konsol çıktısında her model için:

- Accuracy
- Confusion matrix
- Classification report

görüntülenmiştir. Örneğin, model karşılaştırma tablosu şu formdadır (rakamları kendi çıktısına göre doldur):

Model	Accuracy
Linear SVM	0.7917
RBF SVM	0.8750
k-NN (k=5)	0.7583

Ayrıca bu değerler kullanılarak çizilen **doğruluk bar grafiği**, `classification_accuracy.png` dosyası olarak kaydedilmiş ve raporda Şekil olarak kullanılmıştır. Genel olarak, HOG gibi yüksek boyutlu ve çizgisel olmayan ayrımlara sahip özellikler için SVM tabanlı yöntemlerin k-NN'e göre daha kararlı sonuçlar verdiği bilinmektedir; bu çalışmada da benzer bir eğilim gözlenmiştir (hangi modelin en yüksek doğruluğu verdiği, kendi çalıştırdığınız sonuçlara bağlıdır).

4. Analiz ve Tartışma

Bu bölümde elde edilen sonuçlar ve gözlemler özetlenmiştir:

- **HOG'un avantajları:**
 - Kenar ve gradyan bilgisine dayandığı için, aydınlatma değişimlerine belirli ölçüde dayanıklıdır.
 - Özellikle rigid nesnelerin (insan, araç vb.) tespitinde oldukça başarılıdır.

- OpenCV gibi kütüphanelerde optimize edilmiş implementasyonları bulunduğu için pratikte kullanımı kolaydır.
- **Problem 1 (HOG implementasyonu):**
 - Sıfırdan implementasyon, HOG'un nasıl çalıştığını anlamaya yardımcı olmuştur.
 - Parametre değişimleri (hücre boyutu, bin sayısı) doğrudan özellik vektörünün boyutunu ve detay seviyesini etkilemektedir.
- **Problem 2 (Nesne tespiti):**
 - Hazır insan dedektörü, genel olarak makul performans sergilemiş, fakat özellikle arka planın karmaşık olduğu veya insanın kısmen görüldüğü durumlarda hatalar üretmiştir.
 - Özel nesne dedektöründe ise veri miktarı ve çeşitliliğinin sınırlı olması, modelin genelleme kapasitesini etkilemiştir. Buna rağmen, uygun threshold ve NMS ayarları ile belirgin nesneler makul başarıyla tespit edilebilmiştir.
- **Problem 3 (Sınıflandırma):**
 - HOG özellikleri ile eğitilen SVM ve k-NN modelleri, ikili sınıflandırma görevinde anlamlı doğruluk değerleri üretmiştir.
 - RBF SVM genellikle daha esnek karar sınırları ürettiği için daha yüksek doğruluk potansiyeline sahiptir; ancak eğitim süresi Linear SVM'e göre daha uzundur.
 - k-NN'in performansı, kullanılan komşu sayısı ve veri dağılımına oldukça duyarlıdır.

Genel olarak bakıldığında, HOG + SVM kombinasyonunun hem tespit hem de sınıflandırma problemlerinde güçlü bir temel yöntem olduğu görülmüştür.

5. Sonuç

Bu ödev kapsamında, HOG tabanlı özellik çıkarım ve bu özelliklerin nesne tespiti ile sınıflandırma problemlerinde kullanımı detaylı olarak incelenmiştir.

Öncelikle HOG algoritması sıfırdan implement edilerek gradyan hesaplama, hücre histogramı, blok normalizasyonu ve descriptor oluşturma adımları deneyimlenmiştir. Daha sonra OpenCV'nin hazır insan dedektörü ve özel eğitilmiş HOG + Linear SVM modeli ile yaya ve özel nesne tespiti gerçekleştirilmiş, sonuçlar non-maximum suppression ve threshold ayarları ile iyileştirilmeye çalışılmıştır. Son olarak, HOG özellikleriyle çalışan bir sınıflandırma sistemi kurulmuş ve farklı modellerin doğruluk değerleri karşılaştırılmıştır.

Bu çalışma sayesinde hem düşük seviyeli görüntü işleme hem de üst seviye makine öğrenmesi yöntemlerinin entegre kullanımına dair pratik bir deneyim kazanılmıştır. Gelecek çalışmalar için, daha büyük ve dengeli veri setleri, farklı özellik çıkarım yöntemleri (örn. CNN tabanlı özellikler) ve derin öğrenme tabanlı tespit/sınıflandırma yöntemleri ile karşılaştırmalar yapılabilir.

6. Kaynaklar

1. N. Dalal, B. Triggs, *Histograms of Oriented Gradients for Human Detection*, CVPR, 2005.
2. P. F. Felzenszwalb et al., *Object Detection with Discriminatively Trained Part-Based Models*, PAMI, 2010.
3. R. Szeliski, *Computer Vision: Algorithms and Applications*, Springer, 2010.
4. OpenCV Dokümantasyonu – HOGDescriptor.
5. Scikit-learn Dokümantasyonu – SVM, k-NN sınıflandırıcıları.